Project Kickoff Plan

Domain-specific English->Japanese translation and error identification with fine-tuning, RAG, and agentic RAG

Purpose: give the team a concrete build plan that is technically sane, scoped for six people, and aligned with the course brief.

Recommended project statement

We will build a domain-specific English->Japanese translation assistant that produces Japanese translations and flags translation errors. We will compare four system variants: prompt-only base model, fine-tuned model, fine-tuned + RAG, and fine-tuned + agentic RAG. The retrieval layer will use only translation-relevant knowledge assets such as glossary entries, translation memory, style rules, grammar notes, and domain reference snippets.

Assumptions

- One domain only
- Six team members
- Shared repo + reproducible configs
- Limited student compute, so stay in the 7B-8B range
- Goal is a working system plus a clean comparison study

Recommended defaults to lock now

- Backbone: Qwen2.5-7B-Instruct as the default base model. It is Apache-2.0 and its model card explicitly lists Japanese among the supported multilingual languages.
- Fine-tuning: use LoRA/QLoRA, not full fine-tuning. This keeps training feasible on student hardware while still giving you a real fine-tuning contribution.
- System comparison: Base prompt-only -> Fine-tuned -> Fine-tuned + vanilla RAG -> Fine-tuned + agentic RAG. This cleanly isolates each improvement step.
- RAG corpus: glossary + translation memory + style guide + grammar/domain notes. Do not retrieve arbitrary background text that has no effect on translation decisions.
- Error identification output: start with binary error yes/no plus 5 top-level categories; add span-level highlighting only if you have time after the core system works.

1. Why this is a good project frame

Your course brief is asking for an end-to-end GenAI system or study that is technically correct, reproducible, and clearly evaluated. It also explicitly expects meaningful comparisons, quantitative and qualitative evaluation, reflection on what worked or failed, and a reproducible repo/README/config setup. This project frame matches that well because it naturally supports a base-vs-improved comparison study across fine-tuning, RAG, and agentic orchestration.

- It is clearly more than a single assignment: data curation, training, retrieval, orchestration, evaluation, and final demo all connect into one system.
- It supports at least four meaningful alternatives instead of a single one-off model run.
- It gives you both generation quality evaluation and error-identification evaluation, which makes the analysis richer.
- It can be made reproducible with frozen splits, config files, scripts, and a single command path for each experiment.

The main thing to avoid is fake scope: do not build a vague 'translation app with agents' and then realize the RAG documents are irrelevant or the error labels are unusable. The project only works if the external knowledge actually changes translation decisions.

2. Scope freeze: what the project actually is

2.1 Recommended scope

- Task A - Translation: English sentence or short passage -> Japanese translation in the chosen domain.
- Task B - Error identification: given source English and a candidate Japanese translation, predict whether there is an error and classify the error category.
- System variants: Base, Fine-tuned, Fine-tuned + RAG, Fine-tuned + agentic RAG.
- Evaluation focus: terminology accuracy, adequacy/fluency, error-detection quality, and system complexity/latency.

2.2 Hard scope boundaries

- One domain. Do not split across medicine + business + tourism or similar.
- One backbone model. Do not fine-tune three different backbones unless the project later evolves into a model-comparison study.
- One retrieval stack. Keep it stable so your experiment table is interpretable.
- One agentic loop. Do not create a crowd of agents talking to each other unless each step has a clear measurement purpose.

2.3 Domain selection criteria

If the exact domain is not fully locked yet, choose a domain with these properties:

Criterion	Why it matters
Stable terminology	RAG helps only when there are approved terms, naming conventions, or domain rules worth retrieving.
Parallel or near-parallel text exists	You need a realistic fine-tuning set and a human-checkable evaluation set.
Reference documents exist	Glossaries, style guides, FAQs, manuals, policies, or previously approved translations become the RAG corpus.
Ambiguity exists	The domain should contain terms where context changes the correct Japanese translation.
Reasonable annotation load	The team must be able to clean data and label errors without collapsing under manual work.

3. Recommended system design

3.1 The four-system comparison you should implement

ID	Variant	What it does	Why it exists
S0	Base prompt-only	Prompt the base model directly for translation and error identification.	Baseline.
S1	Fine-tuned	Use LoRA/QLoRA adapters on the same base model. Recommended: separate	Measures fine-tuning gain.

		adapters for translation and error identification.	
S2	Fine-tuned + vanilla RAG	Retrieve glossary entries, translation memory examples, and style/grammar rules, then pack them into the prompt before generation.	Measures retrieval gain.
S3	Fine-tuned + agentic RAG	Add one controlled loop: query planning or rewriting, retrieval validation, translation, then self-audit/revision if major issues are detected.	Measures orchestration gain.

3.2 Why RAG makes sense here

RAG is useful for translation only when the retrieved material is decision-relevant. For this project, the retrieval layer should not be generic background knowledge. It should be the exact stuff that a human translator would check: approved terminology, previous translations, style rules, grammar notes, domain references, abbreviations, product names, and similar assets.

- Glossary entry example: 'account settings' -> approved Japanese equivalent, usage note, forbidden variants.
- Translation memory example: previously approved English/Japanese sentence pairs from the same domain.
- Style guide example: plain vs polite register, punctuation rules, katakana usage, UI label capitalization, quote style.
- Grammar/domain note example: how to translate technical compounds, units, button labels, warnings, or user-facing instructions.

3.3 What 'agentic RAG' should mean here

Keep the agentic part narrow and testable. You do not need six agents. A single controlled decision loop is enough.

1. Extract likely key terms or ambiguous phrases from the English source.
2. Retrieve top-k glossary entries, translation memory examples, and style rules.
3. Run a relevance/coverage check: did retrieval find evidence for the key terms?
4. If not, rewrite the query once and retrieve again.
5. Generate the Japanese translation using only the retrieved evidence plus task instructions.
6. Run the error-identification stage against source + candidate translation + retrieved evidence.
7. If a major accuracy or terminology issue is detected, revise once; otherwise return the result.

```
Agentic RAG (recommended minimum version)

English source
 -> term/ambiguity extractor
 -> retriever (glossary + TM + style + notes)
 -> retrieval coverage check
    -> if weak: rewrite query and retrieve once more
 -> translator
 -> error checker / self-audit
    -> if major issue: revise once
 -> final translation + structured error JSON
```

4. Data plan

4.1 The data assets you need

Asset	Format	Target size	Notes
Translation training set	jsonl / csv	Practical target: 3k-10k cleaned in-domain pairs	Used for the translation adapter. Human-clean where possible; otherwise filter aggressively.
Translation dev/test	jsonl	300-500 held-out examples	Must be frozen and human-checked. Split by document/source, not by random sentence only.
Glossary	csv / jsonl	200-1000 domain terms	The most important RAG asset because terminology mistakes are easy to measure.
Translation memory	jsonl	1k-5k examples	Store paired EN/JA examples plus metadata such as topic, tags, and source.
Style guide + grammar notes	md / txt	Small but curated	Short documents are fine if they are specific and actually used by the model.
Error identification set	jsonl	300-500 labeled examples	Use for binary error prediction and category classification.

4.2 Recommended source strategy

- Use real domain material first. Synthetic data can help, but it should not be your entire project.
- Use JParaCrawl as a seed source or weakly supervised source, then domain-filter it and manually inspect samples. It is huge and useful, but because it is built by web crawling and automatic alignment, you should treat it as noisy until cleaned.
- Build your RAG corpus separately from your test set. Do not let evaluation references leak into retrieval.
- Use FLORES-200 only as an optional out-of-domain sanity check, not as the main project benchmark.

4.3 Split hygiene rules

8. Normalize and deduplicate English source sentences before splitting.
9. Split by source document, website, product, or topic cluster where possible. Do not place near-duplicates of the same source text across train and test.
10. Freeze train/dev/test after the first baseline run.
11. Keep a small, manually reviewed 'gold' subset for qualitative examples in the report and demo.

4.4 Error labels: keep them simple

Use MQM-inspired top-level labels, not a giant taxonomy. A practical label set is below. These categories are enough to make the task analyzable without drowning the team in annotation complexity.

Category	What counts
Terminology	Wrong approved term, inconsistent term, or forbidden

	translation variant.
Accuracy	Wrong meaning, omission, addition, or mistranslation.
Fluency/Grammar	Japanese is unnatural, ungrammatical, or awkward.
Style/Register	Wrong tone, politeness level, or style rule violation.
Locale/Formatting	Units, punctuation, script choice, formatting, or locale conventions are wrong.

Recommended output schema for error identification:

```
{
  "has_error": true,
  "severity": "major",
  "categories": ["Terminology", "Accuracy"],
  "rationale": "The domain term is wrong and one source detail was omitted."
}
```

Stretch goal only: span-level highlighting or token-level error marking. Do it only after the core binary/category evaluation is stable.

5. Model and tooling recommendations

5.1 Backbone recommendation

Choice	Model	Why
Recommended default	Qwen2.5-7B-Instruct	Apache-2.0; multilingual model card explicitly lists Japanese; small enough to be realistic for student fine-tuning.
Backup option	Aya Expanse 8B	Strong multilingual focus and Japanese support, but the model card is CC-BY-NC rather than Apache-2.0.
Avoid as default	Llama 3.1 8B	Official model card does not list Japanese among supported languages, so it is not the cleanest default for this project.

5.2 Fine-tuning recommendation

- Use LoRA or QLoRA rather than full fine-tuning.
- Recommended structure: one base model with two adapters - translation adapter and error-identification adapter.
- If your compute is very limited, train the translation adapter first and keep error identification prompt-based until the translation pipeline is stable.
- Store every training run through config files: model name, data version, seed, learning rate, LoRA rank, epochs, batch size, and max sequence length.

5.3 Retrieval recommendation

Use hybrid retrieval. For this task, term exact-match matters a lot, so do not rely on dense embeddings alone. A simple but strong stack is BM25 + multilingual embeddings + optional reranking.

Component	Recommendation

Retriever input	English source sentence + extracted key terms
Dense embeddings	bge-m3 or multilingual-e5-large are both reasonable multilingual starting points
Sparse retrieval	BM25 over glossary entries, translation memory, and style/grammar notes
Chunking rule	Short chunks. Sentence-level or very short paragraph-level. Translation tasks do not benefit from giant chunks.
Top-k	Start with k=3 to 5. More context usually adds noise faster than it adds value here.

6. Evaluation plan

Do not evaluate this only with BLEU. You need task-level metrics, retrieval-level metrics, latency, and qualitative examples. The point is to show what each system change buys you.

Layer	Metric	Why
Retrieval	Hit@k / Recall@k on glossary or translation-memory targets	Tells you whether the needed evidence was actually retrieved.
Translation	COMET + chrF or BLEU	COMET is a stronger MT metric; chrF/BLEU gives a cheap lexical baseline.
Terminology	Term accuracy on a glossary-sensitive subset	Probably the cleanest metric for showing RAG benefit.
Error identification	Binary F1 + macro-F1 over error categories	Shows whether the model can reliably flag and classify errors.
System	Latency, token usage, retrieval time	Shows the cost of RAG and agentic orchestration.
Qualitative review	20-30 manually selected examples by failure mode	Required for honest analysis. Show where each system still fails.

6.1 Recommended experiment table

Comparison	Question answered
S0 vs S1	Does fine-tuning improve in-domain translation and error detection?
S1 vs S2	Does retrieval improve domain terminology and reduce factual/terminology errors?
S2 vs S3	Does the agentic loop improve hard cases enough to justify extra complexity and latency?
Hard subset analysis	Where does each system fail: long sentences, ambiguous terms, multi-term sentences, register-sensitive text, sparse retrieval cases?

6.2 LLM-as-judge use

Use LLM-as-judge only as a secondary rubric-based analysis tool, not as your only evaluation method. A good use case is qualitative scoring on adequacy, fluency, terminology fidelity, and explanation quality of the error checker.

7. Team split for six people

Role	Main responsibilities
1. Project / integration lead	Own repo structure, configs, branch hygiene, experiment sheet, and final merge.
2. Data curation lead	Collect domain data, clean pairs, dedup, build splits, and maintain data documentation.
3. Fine-tuning lead	Train translation adapter, track runs, and create reproducible training scripts.
4. Error-ID lead	Design error taxonomy, label pilot set, train or prompt the error-identification pipeline.
5. Retrieval lead	Build glossary/TM/style corpus, indexing, retrieval evaluation, and prompt packing.
6. Agent + eval + report lead	Implement agentic loop, run ablations, own metrics tables, and coordinate report/demo assets.

Important: these are ownership areas, not silos. Every person still needs visible contribution to the integrated system, and everyone should help produce the final analysis.

8. Timeline and first 10 days

Time	What must be done
Days 1-2	Lock domain, task definitions, label schema, output JSON, and repo structure. Build a 50-example pilot set.
Days 3-4	Run S0 prompt-only baseline on the pilot set. Start glossary and translation-memory collection. Freeze formatting of experiment logs.
Days 5-7	Prepare cleaned train/dev/test data. Train first translation adapter. Build first retrieval index and run retrieval sanity checks.
Days 8-10	Run S1 and S2 on the pilot set. Inspect outputs manually. Fix obvious data or prompt bugs before scaling experiments.

8.1 Week-by-week view

Week	Milestone
Week 1	Data schema, baseline, pilot labels, first glossary/TM version
Week 2	Translation fine-tuning, first metric table, cleanup on bad samples
Week 3	Vanilla RAG, retrieval evaluation, term-accuracy evaluation
Week 4	Agentic RAG, latency analysis, qualitative failure analysis, report + demo

9. Repo structure and reproducibility

```
project/
  README.md
  configs/
    base.yaml
    finetune_translation.yaml
    finetune_error_id.yaml
    rag.yaml
    agentic_rag.yaml
  data/
    raw/
    processed/
    splits/
  kb/
    glossary.csv
    translation_memory.jsonl
    style_guide.md
    grammar_notes.md
  src/
    training/
    retrieval/
    agents/
    prompts/
    eval/
    utils/
  notebooks/
  results/
    metrics/
    qualitative_examples/
    latency/
  demo/
```

- Every experiment must be runnable from a config file rather than handwritten notebook state.
- Fix random seeds where possible and record them in the config.
- Keep data versioning explicit. If the cleaned dataset changes, bump the version and rerun the affected experiments.
- Put prompt templates under version control. Prompt drift can break fair comparison just as much as code changes.

10. Main risks and how to avoid them

Risk	Cause	Mitigation
RAG does not help	RAG corpus is generic or noisy	Restrict retrieval to glossary, TM, style, and rule documents that directly influence translation choices.
Fine-tuning gain is unclear	Train/test leakage or too little in-domain data	Split by source document, dedup aggressively, and inspect a gold subset manually.
Error labels are inconsistent	Too many categories, vague annotation rules	Start with 5 top-level labels, run a 50-example annotation pilot, and write a one-page guideline.

Agentic system becomes spaghetti	Too many steps with no measurement purpose	Keep one loop only: retrieve -> validate -> translate -> audit -> optional revise.
Results are hard to explain	No qualitative review and no subset analysis	Maintain a curated set of good examples and failure examples from the start.
Report feels like disconnected parts	Each member built something separate	Force all work through one experiment matrix and one architecture diagram.

10.1 Anti-patterns to avoid

- Do not let the retriever see the gold reference translation at inference time.
- Do not evaluate on raw noisy web pairs without a human-checked subset.
- Do not use only one automatic metric and call it done.
- Do not present a multi-agent diagram unless each step has an ablation or a clear reason for existing.
- Do not keep changing the domain halfway through the project.

11. Definition of done

Area	Done when...
System	All four variants run end-to-end from script/config and produce structured outputs.
Data	Frozen train/dev/test splits plus documented RAG corpus.
Evaluation	Metric table covers translation, retrieval, error identification, and latency.
Analysis	At least one clear ablation chain: S0->S1, S1->S2, S2->S3.
Qualitative review	Curated examples show success cases and failure cases.
Reproducibility	README explains environment, data prep, training, retrieval indexing, evaluation, and demo steps.
Professionalism	Risks/guardrails documented; team contributions recorded individually.
Demo	A short demo shows at least one case where RAG or agentic RAG changes the translation outcome in a useful way.

12. How this maps cleanly into the final report

Report section	What goes there
Problem & motivation	Why domain-specific EN->JA translation is hard and why error identification matters.
Architecture	One diagram covering S0, S1, S2, and S3 with shared data flow.
Data & models	Dataset sources, cleaning, split rules, glossary/TM/style assets, base model, and adapters.
Experimental setup	Training configs, retrieval configs, prompts, seeds, evaluation metrics, and latency measurement.

Results	Main table plus subset analysis and qualitative examples.
Reflection	What improved, what did not, and which component was not worth the complexity.
Risks & future work	Noise, hallucination, retrieval failure, annotation limits, and next-step improvements.

What the team should do next - immediately

- Lock the exact domain and write one sentence that defines it.
- Agree on the 5 error categories and the JSON output schema.
- Create the repo skeleton and config files before training anything.
- Build a 50-example pilot set and run the base prompt-only model on it.
- Start the glossary first. It will probably become your most valuable RAG asset.
- Do one small fine-tuning run early, even if the data is imperfect. It will expose your pipeline problems fast.

Appendix A. Selected references

These are the main source documents worth keeping in the project README or notes.

Course brief (user-provided PDF): Local assignment PDF provided in chat
Qwen2.5-7B-Instruct model card: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
JParaCrawl: https://www.kecl.ntt.co.jp/icl/lirg/jparacrawl/
FLORES-200 dataset card: https://huggingface.co/datasets/Muennighoff/flores200
COMET MT evaluation docs: https://unbabel.github.io/COMET/html/models.html
Hugging Face PEFT / LoRA docs: https://huggingface.co/docs/peft/package_reference/lora
QLoRA paper: https://arxiv.org/abs/2305.14314
MQM error typology: https://themqm.org/error-types-2/typology/
BGE-M3 model card: https://huggingface.co/BAAI/bge-m3
multilingual-e5-large model card: https://huggingface.co/intfloat/multilingual-e5-large